DECK 13 OF 13 · FRC JAVA CRASH COURSE

Command-based in 2027

Commands v3, side by side with the v2 code you know

WPILib 2027 alpha · Java · Command-based

OVERVIEW

What this deck covers

v3 keeps commands, requirements and triggers. What changes is how you write a command.

- What else changes for command code in 2027.
- The big idea: one function that runs top to bottom.
- Deck 2’s drive and timed autonomous, rewritten in v3.
- Priorities, cleanup and scopes, and two new mistakes to avoid.

THE BIG PICTURE

What changes for command code in 2027

In 2026	In 2027
Commands v2 only	v2 and the new v3. v3 is for Java only.
edu.wpi.first.wpilibj2.command	org.wpilib.command2, or org.wpilib.command3
Java 17	Java 25
motor.set(0.5)	motor.setThrottle(0.5)
SendableChooser for autos	Selectable, published with Tunables
Test mode	Utility mode

Checked against WPILib v2027.0.0-alpha-8. Alpha software, so this can still change.

THE BIG IDEA

Four methods in v2, one function in v3

Commands v2

initialize() runs once, at the start.

execute() is called by the scheduler every 20 ms.

isFinished() is checked every loop.

end(interrupted) runs once, at the end.

Commands v3

One function runs top to bottom, like normal Java.

coroutine.yield() pauses until the next 20 ms loop.

Reaching the end, or return, finishes the command.

whenExited(...) holds cleanup that runs however it ends.

WRITING A V3 COMMAND

A v3 command reads in the order the robot acts

Feeder.java

```
  /** Right: whenExited runs however the command ends. */
  public Command intake() {
1   return run(coroutine -> {
1         motor.setThrottle(FEED_SPEED);
2         coroutine.waitUntil(this::hasPiece);
        })
3       .whenExited(() -> motor.setThrottle(0.0))
4       .named("Intake");
  }
```

1. **run(...)**
   Requires this mechanism. setThrottle is 2027's set.

2. **Wait for a sensor**
   Yields every loop until it is true.

3. **Cleanup**
   Runs however the command ends.

4. **A name is required**
   No name, no command: it won't compile.

FROM V2 TO V3

The v2 words you know, in v3

v2	v3	Note
extends SubsystemBase	implements Mechanism	An interface now
run(() -> …)	runRepeatedly(() -> …)	v2’s run repeats
runOnce(() -> …)	run(c -> …) with no yield	Ends when it returns
.withName(“…”)	.named(“…”)	Required
Commands.waitSeconds(1)	Command.waitFor(Seconds.of(1))	Times use units
.finallyDo(…)	.whenExited(…)	Runs however it ends
CommandScheduler.getInstance()	Scheduler.getDefault()	Same job
.asProxy()	coroutine.await(…)	Built in

WPILib docs: Migrating from Commands v2

COMMON MISTAKE

run means something different

What you see	A drive command copied from v2 moves for one loop, then stops. The command ends at once.
Why it happens	v2's run calls its code every 20 ms. v3's run calls its function once, and a function with no loop ends right away.
The fix	Use runRepeatedly where v2 used run, or write the loop yourself with coroutine.yield().

```
run(() -> drive.arcadeDrive(f, r))           // v2: every 20 ms
run(c -> drive.arcadeDrive(f, r))            // v3: once, then ends
runRepeatedly(() -> drive.arcadeDrive(f, r)) // v3: every 20 ms
```

WRITING A V3 COMMAND

Deck 2’s drive, as a mechanism

KitBotDrive.java

```java
/** Drive from the sticks. Lowest priority, so any other command can take over. */
public Command arcadeDrive(DoubleSupplier fwd, DoubleSupplier rot) {
  return runRepeatedly(() -> drive.arcadeDrive(fwd.getAsDouble(), rot.getAsDouble()))
      .withPriority(Command.LOWEST_PRIORITY)
      .named("Arcade Drive");
}
```

runRepeatedly is v2’s run. The lowest priority lets any other command take the drive.

PRIORITIES

Priorities replace “interruptible or not”

When a new command conflicts with a running one	What happens
The new one has a higher priority	It starts. The old one is cancelled.
They have the same priority	It starts. The old one is cancelled.
The new one has a lower priority	It is discarded. The old one keeps running.

The default priority is 0. Command.LOWEST_PRIORITY suits default commands and idles.

PART 1

Sequences and autonomous

The same routine in v2, in a v3 builder, and as v3 steps

DECK 2, IN V2

The routine students already wrote

deck2/DriveAutos.java

```java
/** The same routine as a list: easier to read, reorder and extend. */
public static Command leaveAndTurn(CANDriveSubsystem drive) {
  return Commands.sequence(
          drive.driveArcade(() -> AUTO_SPEED, () -> 0.0).withTimeout(DRIVE_OUT_SECONDS),
          drive.driveArcade(() -> 0.0, () -> 0.0).withTimeout(PAUSE_SECONDS),
          drive.driveArcade(() -> 0.0, () -> AUTO_SPEED).withTimeout(TURN_SECONDS),
          Commands.print("Auto finished"))
      .withName("Leave and turn");
}
```

Deck 2’s code, unchanged: it compiles against WPILib 2026.

V3, BUILDER

The same routine with v3’s builder

DriveAutos.java

```java
/** The builder: closest to Deck 2's Commands.sequence. */
public static Command leaveAndTurn(KitBotDrive drive) {
  return Command.sequence(
          drive.driveFor(AUTO_SPEED, 0.0, DRIVE_OUT),
          drive.idleFor(PAUSE),
          drive.driveFor(0.0, AUTO_SPEED, TURN))
      .named("Leave and Turn");
}
```

idleFor holds still because KitBotDrive makes its idle drive at zero. The name ends the builder.

V3, COROUTINE

The same routine as plain steps

DriveAutos.java

```java
/** The coroutine: plain Java, one step after another. */
public static Command leaveAndTurnSteps(KitBotDrive drive) {
  return Command.requiring(drive)
      .executing(coroutine -> {
        coroutine.await(drive.driveFor(AUTO_SPEED, 0.0, DRIVE_OUT));
        coroutine.await(drive.idleFor(PAUSE));
        coroutine.await(drive.driveFor(0.0, AUTO_SPEED, TURN));
        System.out.println("Auto finished");
      })
      .named("Leave and Turn (steps)");
}
```

await starts a command and waits for it to end. Any Java fits between steps.

SEQUENCES

Combining commands in v3

Write this	It runs	It ends when
Command.sequence(a, b).named(…)	each one in order	the last one ends
coroutine.await(a)	a, and this waits	a ends
Command.parallel(a, b).named(…)	a and b at once	both have ended
Command.race(a, b).named(…)	a and b at once	either one ends
Command.parallel(a).optional(b)	a and b at once	a ends, and b is cancelled
coroutine.fork(a)	a in the background	this keeps going
a.withTimeout(Seconds.of(2))	a for at most 2 seconds	a ends or the time is up

COMMON MISTAKE

The roller keeps spinning after a cancel

What you see	Bound with whileTrue, the feeder keeps running after the driver lets go of the button.
Why it happens	Cancelling stops the function where it is, inside waitUntil. The line that stops the motor never runs.
The fix	Put stop code in whenExited, which runs however the command ends, as on the first command slide.

```
motor.setThrottle(FEED_SPEED);
coroutine.waitUntil(this::hasPiece); // cancelled while waiting here
motor.setThrottle(0.0);              // so this never runs
```

BEST PRACTICE

Every loop yields

> **Put coroutine.yield(), or a wait that yields, in every loop in a command.**

A loop that never yields freezes the whole robot. WPILib's compiler plugin turns a while loop with no yield into a build error.

```
while (!atTarget()) { drive.arcadeDrive(0.4, 0); }   // build error

while (!atTarget()) {
  drive.arcadeDrive(0.4, 0);
  coroutine.yield();                                 // one loop at a time
}
```

CONTROL THEORY

A feedback loop that reads like the math

Arm.java

```
  public Command holdAt(double target) {
    return run(coroutine -> {
1         pid.reset();
          while (true) {
2           double now = angle.getAsDouble();
2           double ff = KG * Math.cos(now); // gravity
3           motor.setVoltage(ff + pid.calculate(now, target));
4           coroutine.yield(); // one 20 ms step
          }
        })
        .whenExited(() -> motor.setVoltage(0.0))
        .named("Arm Hold");
  }
```

1. **Reset**
   A fresh start for each run.

2. **Measure, then feedforward**
   Gravity, from Deck 6.

3. **Feedback**
   P on the angle error.

4. **One sample**
   Each yield is one 20 ms step: the loop's dt.

SCOPES

Bindings and default commands now have a scope

Made in	Lasts until
The Robot constructor	The robot code stops
autonomousInit, by scheduling	Autonomous ends: the auto is cancelled
An OpMode	That OpMode ends
A running command	That command ends

When a scope ends, its trigger bindings are removed and its default commands are put back.

THE ROBOT CLASS

Deck 2’s bindings and chooser, in v3

Robot.java

```
public Robot() {
  drive.setDefaultCommand(
      drive.arcadeDrive(
          () -> -MathUtil.applyDeadband(driver.getLeftY(), DEADBAND) * DRIVE_SCALE,
          () -> -MathUtil.applyDeadband(driver.getRightX(), DEADBAND) * TURN_SCALE));

  driver.faceDown().onTrue(feeder.intake());

  autoChooser.addDefault("Leave and Turn", DriveAutos.leaveAndTurn(drive));
  autoChooser.add("Leave and Turn (steps)", DriveAutos.leaveAndTurnSteps(drive));
  Tunables.publish("Auto", autoChooser);
}
```

robotPeriodic calls Scheduler.getDefault().run(). autonomousInit schedules the chosen auto.

BEST PRACTICE

Default commands get the lowest priority

> **Give every default command Command.LOWEST_PRIORITY. Raise the priority only for a command that must not be stopped.**

A default command’s priority is the lowest that can ever take its mechanism. Set it high, and button commands are discarded.

CHOOSING

v2 or v3 for your team in 2027

Stay on v2

Same as these decks: only package names change.

Every language: C++ and Python have only v2.

Known patterns: years of examples and team code.

Try v3

Reads like Java: loops, ifs and waits, in order.

Safer by default: required names, yield checks, priorities.

New in 2027: alpha now, so check your libraries support it.

EXERCISE

Port Deck 2 to v3, on paper first

1. Rewrite Deck 2’s driveArcade factory for v3, using the mapping table.

2. Rewrite forwardPauseTurn as a coroutine with coroutine.await.

3. Swap with a partner and check: every command named, every loop yields, cleanup in whenExited.

4. Optional: build it in a 2027 alpha project and fix what the compiler reports.

Done when: both commands are written, and your partner found no run that should be runRepeatedly.

RECAP

What this deck covered

- WPILib 2027 keeps v2 and adds v3, which is for Java only.
- A v3 command is one function: loops yield, return ends it.
- runRepeatedly is v2's run. v3's run runs its function once.
- Names are required, and stop code goes in whenExited.
- Priorities, scopes and await replace interrupt flags and proxies.

REFERENCE

Official docs



Commands v3

docs.wpilib.org (latest)



Migrating from v2

docs.wpilib.org (latest)



v3 design document

github.com/wpilibsuite



v3 HatchBot example

github.com/wpilibsuite